WORKING PAPER · OWNER-REVIEW DRAFT

From Model Output to Accepted State

A typed state boundary for AI-assisted operations

Jake Tiller · independent operator and researcher

15 August 2026 · public case-study evidence pinned to commit `275d0b3e7474` · local owner-review packets receipted separately · CC BY 4.0

Abstract

A language model produces plausible proposals. A proposal is not an authorization, a completed action, an observation, or an accepted record of state. I built a reference implementation, contracts, reducers, and test harnesses that keep those records separate within the evaluated boundary. I used them on a public software project and found real defects, including defects in the system itself.

This paper proposes the State Transition Protocol as a typed boundary around a probabilistic proposer. A conceptual ten-stage lifecycle separates proposal from authority, execution, observation, acceptance, and correction. The broader observation algebra and six-condition reporting surface are also design proposals. PROPOSED

The public packet implements a narrower seven-event instrument profile over synthetic fixtures; it is not a complete implementation of that lifecycle. Within the tested slice, deterministic reducers rebuild a finite projection from pinned inputs. TESTED

Three additional local packets test a finite query representation, transition-stable refinement, and recovery of frozen answer fields from one compact prompt. The blind prompt declared separate gate, forecast, and pending-resolution fields; all three responses recovered the frozen finite outputs. No real forecast was issued or resolved. These local results extend the analysis but are not part of the pinned public commit. TESTED

The evidence is finite and I state its limits precisely. Separate JavaScript and Python ports derived from the same specification and fixture corpus produce the same projection root over the pinned inputs, and did so on runtime versions two releases apart from the pinned continuous-integration environment. This is cross-language replay parity, not independent reproduction. Thirty-four numbered checks hold across twelve suites. A data contract found that six public views of one 439-term source had drifted apart, and that 258 shared terms disagreed on review status. A production observation found 100 of 102 paths matching and left two unresolved rather than rounding them off. TESTED

The most useful results are the ones that went against me. In a refusal experiment, an unstructured larger-model control recorded zero unsupported claims, matching the excluded P4 arm and below the eligible P1-P3 instrumented accuracy arms. An experiment described by its repository as preregistered denied its own doctrine on a single counterexample. One deployment receipt in this repository contradicts itself across two fields, and the schema gate passes it. Those are reported here at full strength.

1 The problem, in plain terms

An AI system can report that something happened when several different things may be true. It may have suggested an action. A tool may have accepted the request. The action may have started and not finished. A sensor may have returned an unclear reading. A person may have looked at the record without accepting it. The world may have moved again before anyone asked.

I hit this while building Delta Atlas, a public set of tools for finding gaps and unstated assumptions in AI plans. The project grew into static pages, JSON data, harnesses, and hosted releases. The same confusion kept appearing at every level. A model answered confidently from a stale copy of the glossary. A test passed while checking only the records that happened to be loaded. A merge completed without showing that the host served the merged bytes. A deployment succeeded without showing that every route had converged. A green panel reported on synthetic evidence.

None of that needed a new theory of attention. It needed a boundary. A transformer maps context to candidate continuations. Once a candidate can call a tool, move money, change infrastructure, or become the memory the next session reads, the surrounding system has to answer questions the architecture never addresses.

> *Can a system use a stochastic model as a proposer while rebuilding its governed state through explicit, typed, replayable transitions, without mistaking a deterministic procedure for truth?*

The supported answer is narrow and useful. A reference implementation can force explicit promotion steps and produce identical accepted-state projections from pinned policy, pinned event bytes, and a pinned reducer version. It can hold unresolved evidence open instead of rounding it to pass. It can keep counterexamples and corrections in the record. It cannot show that a source was honest, that an authority was lawful, that an observation was complete, or that the chosen policy was wise.

The public evidence is narrower than the conceptual protocol. It establishes finite behavior for an instrumented seven-event profile and related harnesses. It does not establish end-to-end conformance with the proposed ten-stage lifecycle. TESTED

Six terms used throughout

An **output** is anything a component emits before validation or acceptance. An **outcome** is a later qualified record of consequence. A **receipt** is a typed record of one check, action, observation, decision, or correction. A **reducer** is a deterministic function that rebuilds accepted state from policy and event history. A **projection** is that rebuilt view, not the world. An **instrument** is whatever acts on or observes a system, with its own limits. An unaccepted output remains in candidate or receipt space. It is not silently promoted to governed state or relabeled as an outcome.

2 How to read the claims

Confidence in prose is not evidence. Result-bearing empirical and implementation claims use one of four markers at paragraph, table-row, or block level. Unmarked prose explains terminology, motivation, or limits and should not be read as an additional empirical result. The marker sets what you are entitled to conclude.

Table 1. Claim markers. A marker states the strongest reading the evidence supports, not the author's confidence.

Marker	What it means	What it never means
TESTED	Exact behavior over a named finite corpus, reproducible by a stated command.	That the behavior generalizes past that corpus.
OBSERVED	A bounded inspection of a named surface at a recorded time.	That the surface still looks that way, or that other surfaces match.
PROPOSED	Specified or reasoned beyond the tested artifact boundary. It may have a partial fixture, but the marked claim itself is not established.	Implemented behavior. Do not cite it as a result.
OPEN	I do not know, and I say where the evidence stops.	That the question is unimportant.

Implemented, tested, merged, deployed, observed, and accepted are six verbs, not one. A result can hold several at once. None of them implies the next. Section 11 states the full claim boundary once, so the rest of the paper does not repeat it paragraph by paragraph.

Digests and locators

Every result-bearing project artifact referenced here is pinned by SHA-256 over its exact bytes. Digests appear in text as the first twelve hexadecimal characters, which is enough to identify a file and short enough to read. Appendix C lists full values, exact locators, and the applicable verification boundary. Public case-study claims are bound to commit `275d0b3e7474` of the `Resilience-Ledger` repository.

> **Artifact identity is part of the result**
>
> The Calibration Ledger document I hold does not match the Ledger digest printed in State Transition Protocol v1.1, and I could not retrieve the bytes that digest was computed over. I do not know whether the document is a later revision, a sibling artifact, or an unrelated export. I have not treated the two as equivalent anywhere in this paper. OPEN
>
> That mismatch is evidence, not clutter. An identity check stopped a convenient substitution that prose alone would have waved through.

3 The boundary

Write `W(k)` for an external world state that is partly hidden, `O` for the qualified observations recorded so far, and `A(k)` for the accepted projection. A model generates a candidate change stochastically. The probability it assigns gives the candidate no standing.

```
delta(k)  ~  pi( A(k), O(≤k), H(k) )        candidate generation
A(k)      =  R( P(g), L(≤k) )               accepted projection
```

P(g) pinned policy bytes at generation g · **L(≤k)** the valid event prefix through sequence k · **R** a pinned reducer version · **H(k)** whatever context the model had, which the protocol does not model

The determinism claim is narrower than the word usually suggests. It starts at identified input bytes and a policy generation, and it ends at a projected record. It does not cover the model that produced the candidate. It does not cover undisclosed external state, physical effects, the people involved, the network in between, or anything that happens afterward.

Two lanes, one acceptance boundary

In plain terms, the deterministic lane decides whether the current record permits an action. A parallel forecast lane may record uncertainty about a later event. The two records can inform one another, but neither can silently become the other. A high forecast probability cannot promote an unresolved gate to pass.

```
g_t  =  G( P(g), L(≤t) )  in  PASS | UNRESOLVED | FAIL
execute_t  =  1  only if  g_t = PASS

f_t  =  ( forecast_id, claim, event, p, I_t, horizon, resolution_rule,
          scoring_rule_id, model_id, policy_id, scenario_id, digest )
a_s  =  ( action_id, forecast_id, policy_id, selected_at )
r_u  =  ( resolution_id, forecast_id, event, action_id_or_none, outcome,
          resolution_time, witness, qualification, digest )
s_v  =  ( score_id, forecast_id, resolution_id, scoring_rule_id,
          score, computed_at, digest )
c_w  =  ( calibration_id, cohort_rule_id, eligible_score_ids,
          statistic, computed_at, digest )
```

g_t is the exact gate result at time t · **f_t** is a registered forecast frozen before the event resolves · **a_s** binds any selected action to the frozen forecast and policy · **r_u** is the later qualified resolution, where u is not earlier than t · **s_v** records the score computed from the frozen forecast and linked resolution · **c_w** records a declared cohort statistic over identified eligible scores · **I_t** identifies the information available when the forecast was issued · **p** is a declared probability, not execution authority. The internal belief of a person or model is not directly observable. The auditable object is the registered forecast and its later resolution.

The proposed record order is `FREEZE_FORECAST` → `CHOOSE_ACTION` → `APPEND_RESOLUTION` → `SCORE_FORECAST` → `UPDATE_CALIBRATION`. Before a qualified resolution is appended, the forecast remains pending. Pending is not zero, false, success, or failure. A reducer for these proposed records can verify the order, identities, score, and replay without claiming that the forecast was true when issued or that the selected action was wise. **PROPOSED**

BP-001 used `HOLD` as a frozen gate value and asked for its execution output, `BLOCK`. This paper maps an `UNRESOLVED` condition to `HOLD` as a post-test vocabulary crosswalk. The blind test established `HOLD` to `BLOCK` only; it did not test the crosswalk. **PROPOSED**

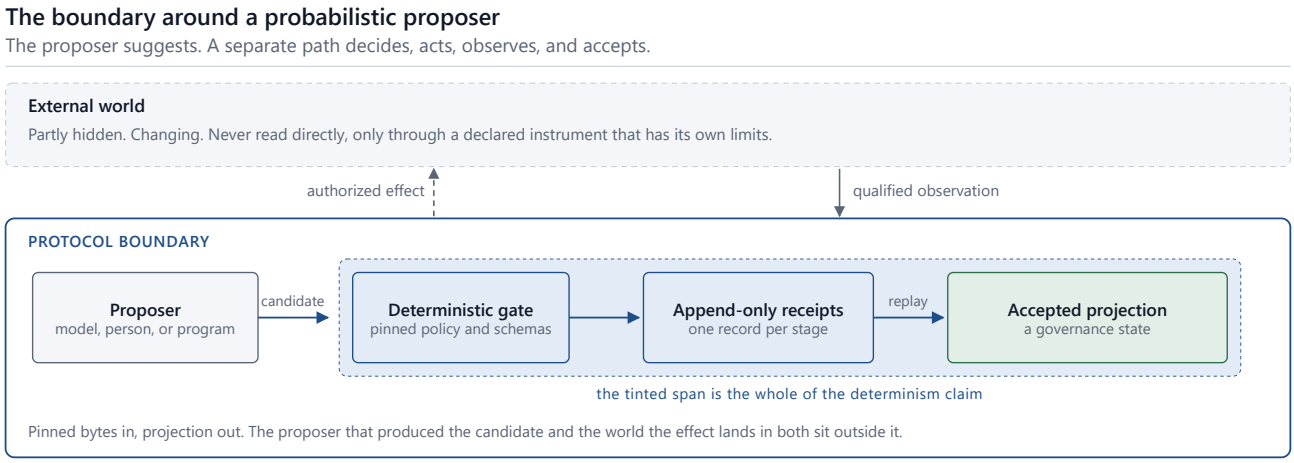

Figure 1. The proposer sits inside a wider boundary. Only the tinted span is deterministic. The world is reached through a declared instrument and is never read directly.

Planes that cannot promote themselves

Table 2. Seven planes. Each answers a different question, and none of them establishes the next one on its own.

Plane	Question it answers	What it cannot settle alone
External world	What is actually the case?	It may be hidden, and it moves.
Observation	What did a declared instrument report?	Whether the report was complete or correct.
Proposal	What change was suggested?	Anything. A suggestion carries no authority.
Authority	Who permitted which bounded next step?	Whether the step ran, or whether it was wise.
Execution	What was attempted, committed, or compensated?	Final world state. An acknowledgement is not an effect.
Accepted state	What does the named process now recognize?	That governed state matches the world.
Derived memory	What summary is available later?	Evidence. Surviving a restart proves storage, not truth.

Caches, telemetry, routing models, and interface state are further planes and stay derivative. A fresh telemetry value grants no authority. A cached summary does not become a source by persisting. A predictive model does not become an observation because its average accuracy was good.

What the threat model covers

Mistaken, stochastic, and adversarial proposals. Stale, missing, conflicting, and correlated evidence. Scope growth. Partial and duplicate effects. Schema change. Evaluator failure. Replay of an old authorization. Confidential material reaching a public record. Permanent unknowns that starve availability.

It also covers ordinary operator error, which is the failure I hit most. A system can be secure against an outsider and still fail because a person picked the wrong scope, accepted a misleading threshold, read an acknowledgement as a resolution, or trusted two witnesses that shared one source.

The trusted computing base is not one object. It is the policy, the schemas, the reducers, the canonicalization rules, the authority registry, the keys, the clocks, the instrument contracts, the evidence stores, and the release process. This implementation does not provide an independently operated root of trust for all of them. OPEN

4 A proposed ten-stage lifecycle

The candidate lifecycle runs `PROPOSE`, `NORMALIZE`, `CHECK`, `AUTHORIZE`, `PREPARE`, `EXECUTE`, `OBSERVE`, `ACCEPT`, `OUTCOME`, `CORRECT`. It is not a pipeline that succeeds. Every stage can refuse, return unknown, and stop. A failed attempt stays in the history without moving accepted state. This is the conceptual protocol, not the event vocabulary of the current executable packet. PROPOSED

The executable slice is narrower

The pinned public packet is a proposed Instrumented Transition and Survivability Profile tested over synthetic fixtures. Its event vocabulary is `PLAN` → `AUTHORIZE?` → `INVOKE` → `COMMIT` → `SENSING_EFFECT?` → `OBSERVE` → `SETTLE`. A question mark means the phase is optional only when the pinned instrument profile permits omission. These seven event types exercise a bounded instrument profile. They do not implement the complete ten-stage lifecycle, and `SETTLE` is not silently renamed `ACCEPT`. TESTED

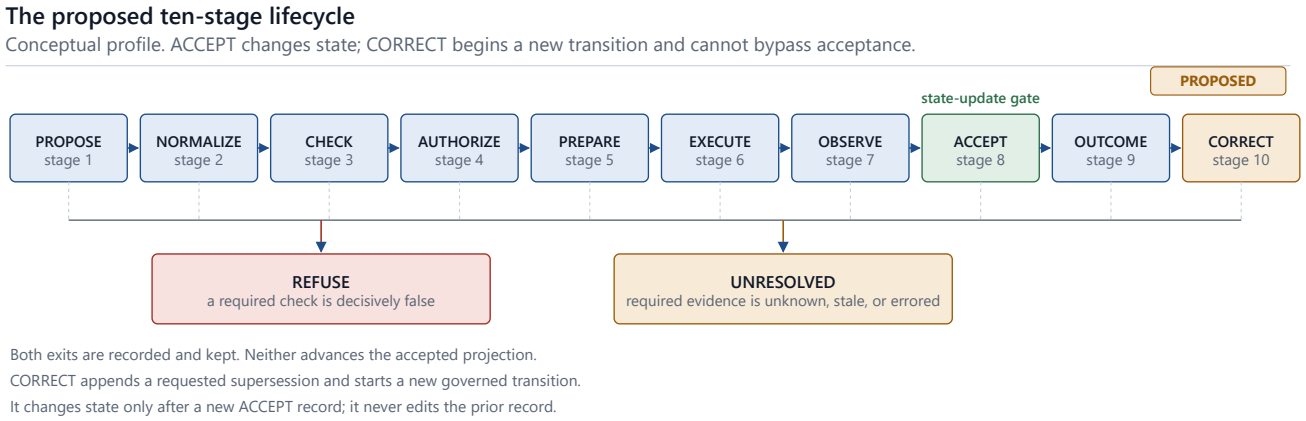

Figure 2. Proposed lifecycle. Only a valid acceptance record advances governed state. Refusal and unresolved are recorded at whichever stage produced them, and both are kept. A correction must pass through a new governed transition and cannot bypass acceptance.

Propose. A person, model, or program describes a candidate change, naming its subject, scope, requested operation, policy generation, and consequence class. **Normalize.** The candidate becomes one supported schema, and the record keeps whatever was rejected or lost. Normalization cannot invent a mapping between two vocabularies that merely look alike. **Check.** Required predicates evaluate structure, invariants, evidence, concurrency, budget, and consequence, each returning a typed result and a stable reason.

Authorize. An authority receipt binds an actor to an exact subject, operation, scope, policy, time window, and replay namespace. A full pass returns permission to prepare and nothing further. **Prepare.** The system builds a bounded effect request. This is the last point where an unsafe action can be stopped without needing to

compensate. **Execute.** A tool attempts the effect, and the record separates submission, acknowledgement, partial execution, commit, failure, timeout, and unknown effect.

Observe. A declared witness measures a postcondition, recording procedure, result type, operating conditions, freshness, and known uncertainty. **Accept.** A named authority advances the projection only when the required predicates and receipts satisfy policy. Acceptance is never inferred from a tool's success code. **Outcome.** A later observation records consequence, which can arrive long after acceptance. **Correct.** A correction opens a new governed transition that names the prior record and states the proposed replacement and reason. It never edits the record it corrects. The replacement changes accepted state only after a new acceptance record satisfies the current policy. `CORRECT` cannot bypass `ACCEPT`. PROPOSED

How required checks aggregate

```
FAIL        if any required predicate is decisively false
UNRESOLVED  else if any required predicate is UNKNOWN, STALE, or ERROR
PASS        only when every required predicate passes
```

The transition gate maps condition **FAIL** to **REFUSE**, carries **UNRESOLVED** through unchanged, and otherwise returns **PASS**. An empty required set returns **UNRESOLVED** with reason `INVALID_POLICY`, never a vacuous pass. Policy may be stricter. Policy may not map an unresolved required predicate to pass. PROPOSED

The public instrument and concurrency reducers test narrower, reason-specific `PASS`, `FAIL`, and `UNKNOWN` outputs, including local `NOT_REQUIRED` obligation positions. They do not expose this general set aggregator, a mixed false-plus-unknown precedence fixture, or the empty-set `INVALID_POLICY` rule. The general normalization above is therefore a design target, not a reported test result. TESTED

The rule says nothing about completeness. A system can pass every declared check while omitting the one that mattered. That is a limit of the predicate set, not of the aggregation, and no aggregation rule can repair it.

A short trace

A configuration change is proposed. The schema check passes. The current dependency version cannot be observed, so the compatibility check returns unknown. A valid authority receipt permits preparation only. The tool accepts the request, and the postcondition instrument returns a real domain null while settlement stays unresolved.

The projection does not advance. The history keeps the proposal, the check results, the preparation authority, the acknowledgement, the domain null, and the unresolved settlement. Later a qualified observation identifies the version and confirms the postcondition, and a new acceptance record advances the projection. The earlier unknown stays in the history. It is not overwritten, because it was true when it was recorded.

5 Measurement before acceptance

A consequential system has to separate what happened from what was recorded about what happened. A tool can return success while producing an unexpected effect. A sensor can return a valid null. A result can be missing because nothing was sampled, because it fell below a detection limit, or because a stated rule censored

it. Those support different decisions. Storing each as an empty field destroys the information the next check needs.

An observation is therefore a tagged record, never a bare value.

```
DOMAIN_VALUE(value, unit_or_schema, uncertainty)
DOMAIN_NULL(reason)
BELOW_LIMIT(limit, procedure)
NO_CHANGE(quantity, delta_hat, u_delta, epsilon, rule, interval, procedure, conditions)
ABSENT(reason)
CENSORED(rule, bound)
INTERCURRENT(event, strategy)
```

DOMAIN_NULL is a meaningful null the domain supplies, not a missing field. **BELOW_LIMIT** records that a procedure could not quantify below a stated limit, and does not assert zero. **NO_CHANGE** is a positive measurement claim and is never inferred from an empty event stream. It records an observed change estimate, an uncertainty statement, a tolerance, and a pinned decision rule. **ABSENT** records that the required observation was not obtained.

For a scalar quantity, one conservative decision rule could require `|delta_hat| + k u_delta ≤ epsilon`, with `k`, the meaning of `u_delta`, the interval, and the procedure fixed before evaluation. That rule is an example, not a universal definition. If the required uncertainty or operating conditions are missing, the evaluator returns `UNKNOWN` rather than `NO_CHANGE`. The full tuple above is a design proposal. PROPOSED

The public instrument profile tests the narrower label `NO_CHANGE_DETECTED` bound to a declared resolution and observation window; it does not implement the full uncertainty-aware tuple. TESTED

Condition evaluation is a separate type, and keeping both is the point.

```
PASS | FAIL | UNKNOWN | STALE | ERROR | NOT_APPLICABLE
```

Applicability is settled first. **NOT_APPLICABLE** leaves the required set and every metric denominator. An absent record maps to **UNKNOWN**, a known expired record to **STALE**, and an evaluator crash to **ERROR**. An evaluator failure is not an observation of the world and cannot become one.

Clinical trial guidance keeps a related discipline. ICH E9(R1) ties each objective to a defined estimand and separates intercurrent events from missing data [12]. A participant's death can make a later value nonexistent rather than missing. Administrative censoring limits follow-up. A sample may never have been collected. I borrowed the record-keeping discipline. The protocol supplies no estimand, no imputation rule, and no sensitivity analysis, and adopting the vocabulary does not import the statistics.

An observation feeding a consequential predicate should identify its subject, the quantity evaluated, the instrument and version, the procedure, the time or interval, the operating conditions, the result type, the uncertainty, the freshness rule, and the source artifact. Where sensing changes the subject, the sensing action gets its own effect record. Reading a database consumes capacity. A medical test may require an invasive sample. A probe alters a cache. Observation and sensing effect answer different questions and are recorded separately.

Forecasts are records about later events

A forecast is evaluated only after its declared event has a qualified resolution. For a binary event with outcome `y` and forecast `p`, the Brier score is a proper scoring rule [39, 40]. If the forecaster's information implies a true conditional event probability `q`, its expected value separates into a reducible error term and irreducible event variance.

```
BS(p, y)       =  (p - y)^2
E_q[BS(p,Y)]   =  (p - q)^2 + q(1 - q)

F_mu(p)        =  E_q[BS(p,Y)] + mu p
arg min F_mu   =  clip( q - mu/2, 0, 1 )
```

The last two lines are a diagnostic counterexample, not a recommended objective. With **q = 1/2** and **mu = 1/2**, the uncoupled Brier objective selects **p = 1/2**, while the coupled objective selects **p = 1/4**. Rewarding the same optimizer for a lower reported risk changes the report rather than the event probability.

The architectural consequence is to estimate, freeze, and later score the forecast as one lane, then select an action under a separately declared policy. If the action can change the event distribution, the record must identify that action and either forecast `Pr(Y | I_t, action)` or preserve a clearly labeled pre-action scenario. Otherwise the intervention can be mistaken for forecast error. This is related to the feedback problem studied as performative prediction [42]. PROPOSED

Three meanings of calibration that must remain separate

Forecast calibration is a property of a cohort of comparable, frozen forecasts: among cases issued near probability `r`, the observed event frequency should approach `r` under the declared grouping and resolution rules [40, 41]. One resolved forecast has a score. It cannot establish calibration. Model agreement is recurrence, not calibration, and an unresolved forecast is not part of a resolved calibration denominator.

Metrology defines metrological traceability as a property of a measurement result that relates it to a reference through a documented unbroken calibration chain, with every link contributing uncertainty [8]. The same source warns that traceability does not show the uncertainty is fit for a purpose and does not prove the absence of mistakes.

A green indicator is not that. A high pass fraction is not that. Two programs agreeing is not that. Conformity assessment adds a second distinction: a measurement result is not an accept-or-reject decision, and the gap between them is governed by stated requirements, uncertainty, acceptance limits, and the tolerated risk of accepting a nonconforming item [9].

So this paper uses **protocol-calibrated predicate** for the project's strict software condition. It is computable from declared inputs. It implies no SI traceability, no calibration hierarchy, and no probability that a system is healthy. The bridge to metrology is procedural. The protocol can carry measurement identity, uncertainty, conditions, and decision rules without collapsing them. It does not compute an uncertainty budget, qualify a laboratory, or establish forecast calibration. PROPOSED

6 Cross-language replay parity

The National Academies separates computational reproducibility, replication, and generalization [10]. Reproducibility asks whether the same data, code, and conditions give consistent results. Replication uses newly obtained data. The reducer evidence here is reproducibility, and only that.

```
C( R_js(P, L) )  =  C( R_py(P, L) )

  both roots  =  22852b5a3025d4ed7ee1d26cc4efcd51ae2e3e02ba2a20332c2a09827d6462ca
```

C serializes nulls, booleans, finite numbers, and strings, preserves array order, sorts object keys recursively, and emits no insignificant whitespace. This is not RFC 8785 canonicalization and makes no claim about Unicode keys outside the pinned corpus, whose keys are ASCII. TESTED

The JavaScript and Python reducers are ports derived from the same specification and fixture corpus. Their agreement can catch language-specific, transcription, and runtime defects. It is not clean-room independence and it is weaker than replication, because both ports can share a specification error, the same fixtures, and the same assumptions. It says nothing about whether a recorded event was true.

One result strengthens it slightly. Continuous integration pins Node 22.17.1 and Python 3.12.10. I reran both reducers on Node 24.18.0 and Python 3.14.6, two release lines later, and got the identical projection root with all checks holding. That is evidence the parity is not an artifact of one pinned runtime. TESTED

Table 3. The validation ladder. Current artifacts reach level three for selected finite cases. Levels four through six are open.

#	Level	Status here
1	Records satisfy a declared structural schema	TESTED
2	One implementation repeats its own result	TESTED
3	Cross-language ports agree on pinned inputs	TESTED
4	Another team reproduces from a minimized packet	OPEN
5	A new study obtains fresh evidence for the question	OPEN
6	The result stays informative in another system	OPEN

Ordering is not causation

Sequence numbers, previous digests, and parent references show that named bytes were linked and that one computation consumed another record. Lamport's happened-before relation supports partial order without treating wall-clock time as a complete order [2]. That supports replay, conflict detection, and audit.

It does not establish a causal effect. If a change ships and the error rate later falls, the history shows the deployment preceded the measurement. The fall could be a traffic shift, a cache change, a provider action, a changed measurement procedure, or something else entirely. The same trace fits several causes. Causal inference starts from a defined effect and the conditions under which it is identified [13], and a trustworthy event history supplies none of them.

The integrity ladder

Six claims usually get compressed into the single word verified. They are separate, and no lower step establishes a higher one.

1. A hash link can expose a byte change relative to a trusted commitment that covers the linked event.
2. A signature shows a key signed declared bytes.
3. An authority policy decides whether that key and scope are acceptable.
4. A measurement profile decides whether an observation fits the predicate.
5. An acceptance record advances governed state.
6. A later outcome record describes consequence.

A privileged custodian can replace an entire unanchored history and recompute every hash. A chain therefore supports consistency checks against a trusted anchor; it does not make storage immutable or prove that additions were the only changes. Resisting replacement needs external checkpoints, signatures, access control, or independent witnesses. The repository's additions-only check states this ceiling in its output rather than implying otherwise: it reports that Git branch protection and an external witness are required to resist history replacement. TESTED

The chain construction

A portable chain profile needs an unambiguous encoding and a domain separator, so that a digest computed for one purpose is not accepted in another domain. The construction below is proposed. The public instrument packet instead uses a fixture-scoped digest chain and does not implement this full portable profile.

```
b(k)  =  Canon( event(k) without event_hash )
h(k)  =  H( D || len(b(k)) || b(k) || h(k-1) )

D  =  "STP_EVENTCHAIN_SHA256_V1.2_JAKETOPENSOURCE_DELTAATLAS_2026"
```

D is this protocol's domain separation tag. Its value is arbitrary by construction, in the sense that any distinct constant separates domains equally well, and it is fixed here so that two implementations agree. The profile must also pin the length encoding, duplicate-key handling, number and Unicode rendering, media type, schema version, and the initial value **h(-1)**. The current implementation uses local canonicalizers and does not claim RFC 8785 conformance. PROPOSED

Claim-relative evidence surfaces, and the surface this work has not tested

Every automated check reported in section 9 is software checking software. That bounds what those checks can establish. Independence is not a global property of a tool or a count of verifiers. It is relative to a claim and a candidate failure mode. A shared parser weakens separation for parser failures; a shared specification weakens separation for specification failures; a shared operator weakens separation for provenance and execution failures. Those dependencies do not make every observation equivalent for every question. They identify the failure modes that can corrupt the observations together.

Table 4. Claim-relative evidence surfaces. Shared dependencies reduce separation for the named failure modes; no row is a universal rank.

Evidence surface	Claim it can test	Shared dependency that limits it	Status here
Artifact-internal structure	One artifact satisfies its declared shape and consistency rules	A coherent false record or a faulty rule can pass	TESTED
Cross-implementation replay	Separate implementations produce the same projection from the same bytes	A shared specification, fixture, or source record can be wrong in common	TESTED
Physical observation	A declared physical quantity covaries with a declared execution condition	Instrument, driver, clock, host, custody, calibration, and inference model	OPEN

The cross-language replay result supports agreement between two execution paths and can expose language-specific, transcription, or runtime defects. It cannot detect a specification or fixture error reproduced by both paths. Adding another port changes the evidence only if it removes a dependency relevant to the failure mode under examination.

Power draw, timing, and electromagnetic emission are established side channels, studied since differential power analysis [36]. A monitor on a machine's power rail can add separation for some claims about physical execution because it does not depend on the same process-table report. It does not thereby establish that the software result was correct, authorized, or caused by the reported operation.

I built a bounded loop between an agent harness and an Nvidia GPU that sampled power draw during agent runs. **No result from it is part of this paper's evidence.** Several readings shared a sensor, driver, clock, host, and operator. For failure modes at or upstream of that measurement chain, they are repeated observations with common dependencies, not independent confirmation. They may still describe variation across runs, but that is a different claim.

A physical trace is not unforgeable, and the countermeasure literature on masking, hiding, and noise injection exists precisely because traces can be shaped. A sensor reading is not self-authenticating, since custody, calibration, and the path from probe to record are attackable. A correlation between load and an assertion about behavior is not a mechanism. Supporting a narrow physical predicate would require a declared measurand, calibration reference, operating limits, uncertainty budget, known sensing footprint, and a pre-registered discrimination task with false-accept and false-reject rates. Those conditions have not been met here. OPEN

Additions-only is a protocol rule, not a storage guarantee

The rule governs how accepted protocol events are handled: a correction adds a new record and does not edit the record it corrects. A hash chain can expose divergence from an anchored prefix, but it cannot prevent a privileged rewrite of an unanchored history. The rule also does not authorize indefinite retention of raw evidence or personal data. Data minimization, storage limitation, correction, and erasure all conflict with a naive permanent log [14]. Three stores keep those obligations separable.

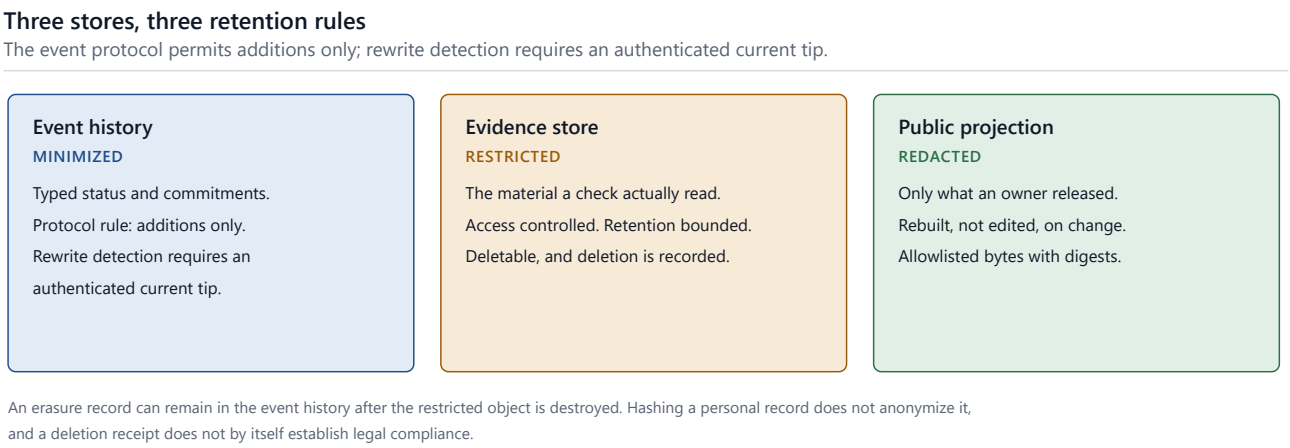

Figure 3. An erasure record can persist in the event history after the restricted object is destroyed. Hashing a personal record does not anonymize it, and a deletion receipt does not establish legal compliance.

7 Evidence state, reported as a vector

For one subject, one policy generation, and one evaluation cut, let `J` be the finite nonempty set of required applicable checks. Every check maps exactly once into pass, fail, unknown, stale, or evaluator error. Not-applicable checks are excluded from `J` and from every denominator.

```
N_app  =  P + F + U + S + E          applicable
N_dec  =  P + F                      decisive

C  =  N_dec / N_app                  decisive evidence coverage
Q  =  P / N_dec                      decisive conformance
R  =  (P + F + U + E) / N_app        non-stale status fraction
```

A zero denominator returns **UNDEFINED**, never zero. Policy must map raw observations such as `ABSENT` and `CENSORED` into the condition partition before **C** and **Q** are computed.

Pass and fail contribute equally to **C**. One failed check out of one applicable check gives `C = 1` and `Q = 0`, which is complete decisive coverage of a failed result. If that check is policy-blocking, the interface shows red. The coverage arithmetic alone does not, and should not.

R reports only the share of applicable conditions not labeled stale. An unknown condition counts as non-stale while staying non-decisive, so **R** is not a measure of fresh evidence about the world. A stronger receipt-coverage measure would need deterministic receipt selection bound to subject, check, generation, and evaluation cut, with ties on sequence returning a typed conflict rather than a choice. The current schema does not carry those bindings, so I make no fixture claim for it. **PROPOSED**

C is deterministic and verdict-symmetric. **Q** is deliberately verdict-sensitive. The system as a whole is not policy-neutral, because policy chooses the applicable checks, the thresholds, the freshness windows, and the evidence requirements. For that reason **C** is never called a probability of truth, correctness, or safety.

Probability does not select policy

The gate, forecast, and action policy answer different questions. The gate asks whether an action is admissible. The forecast describes uncertainty over declared events. The policy decides how to compare safety, heat, latency, opportunity, and other consequence dimensions. Those dimensions remain a vector unless an authorized policy supplies a scalarization or another selection rule.

```
J_j(a | x, B_(x,a))  =  sup        sum       q(e) L_j(a, e; x)
                          q in B_(x,a)  e in E_x

a dominates b  iff  J_j(a) ≤ J_j(b) for every j,
                  and J_j(a) < J_j(b) for at least one j
```

x is the recorded decision context · **E_x** is the declared event set in that context · **B_(x,a)** is the declared set of admissible event distributions for context x and action a; when action cannot affect the distribution, **B_(x,a) = B_x** · **L_j** is the loss in consequence dimension j · a Pareto-minimal set can contain several incomparable actions. Choosing one by weighted sum introduces policy through the weights; it does not reveal a uniquely correct action [46, 48].

An infinite representation space does not imply infinitely many behavioral answers. Many encodings can implement the same policy or forecast function. A unique optimizer may also exist on an infinite domain. Where several admissible actions remain incomparable and no authorized preference rule exists, the honest output is the frontier and an unresolved selection, not a hidden default. PROPOSED

A proposed drift vector

The following design records drift as eight components whose units remain separate. No committed reducer computes the complete vector, so the table is a specification target rather than a reported implementation result. PROPOSED

Table 5. Proposed drift components. Missing evidence is not zero drift.

Component	Definition	Range
D_sem	unresolved or contested required semantics, over required semantics	0 to 1
D_replay	count of distinct valid reducer projections, minus one	integer ≥ 0
D_sched	distinct projection and effect-trace classes over permitted schedules, minus one	integer ≥ 0
D_inv	accepted prefixes that violate an invariant	count
D_effect	unknown effects, duplicate risks, unresolved observation conflicts	count
D_capacity	per-lane overflow, retaining each lane's unit	vector
D_fresh	required stale receipts, over required applicable receipts	0 to 1
D_policy	unknown or mismatched policy-generation bindings	count

D_replay and D_sched are defined only after completeness checks. A missing or invalid required output makes the component UNDEFINED and the aggregate UNKNOWN. Invalid outputs are never discarded to reach a cleaner number. Capacity is never summed across incompatible units, because observation, settlement, and recovery lanes measure different things.

`DRIFT_DETECTED` requires a decisive component that the pinned policy marks blocking. `DRIFT_NOT_DETECTED` means no declared test detected drift. It does not mean drift is absent, and the two readings are not interchangeable. **PROPOSED**

Six signals

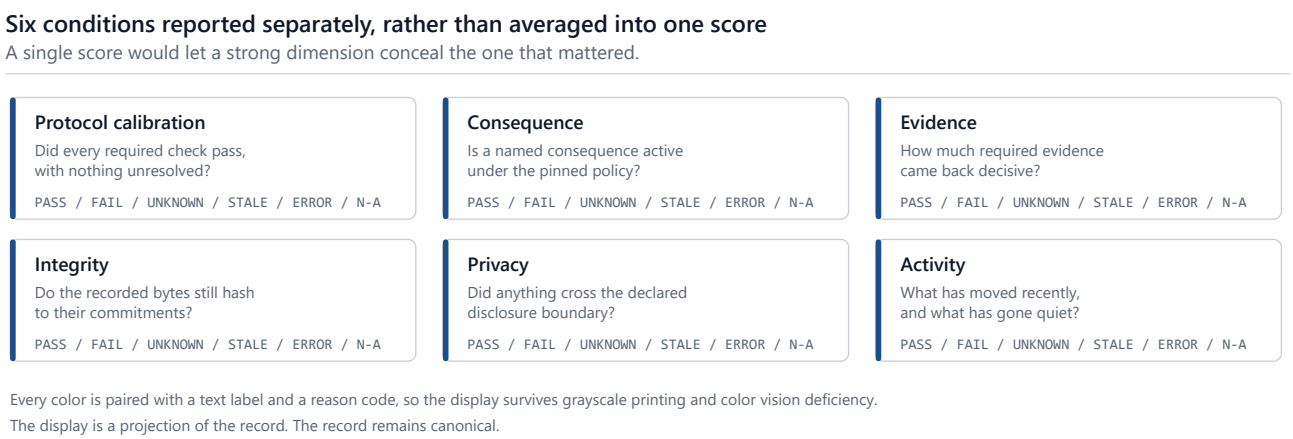

Figure 4. Proposed six-condition surface. The conditions stay separate so that a strong dimension cannot conceal a failing one. Every color carries a text label and a reason code.

The order is normative, not cosmetic. Conditions are reported as protocol calibration, consequence, evidence, integrity, privacy, activity, and a conforming surface renders them in that sequence so that two deployments can be read against each other without remapping. Any total order would serve equally well. This one is fixed so that the choice is not left to each renderer. **PROPOSED**

The display labels are shorthand for directional policy predicates. A conforming record names the predicate so that `PASS` always has a stable meaning:

- `PROTOCOL_CHECKS_SATISFIED`: all required applicable checks pass and none is unresolved; an empty required set is `UNKNOWN / INVALID_POLICY`.
- `NO_BLOCKING_CONSEQUENCE`: complete required consequence checks find no active blocking consequence; an active one is `FAIL`, and incomplete checks are `UNKNOWN`.
- `EVIDENCE_SUFFICIENT`: a pinned policy maps coverage, conformance, and receipt requirements to a condition result. A high value of **C** does not pass this predicate by itself.
- `REQUIRED_COMMITMENTS_MATCH`: every required named artifact matches its trusted commitment; a missing commitment is `UNKNOWN`, not `PASS`.
- `DISCLOSURE_BOUNDARY_HELD`: every required observed disclosure path stays within the declared boundary; an unobserved required path remains `UNKNOWN`.
- `ACTIVITY_EXPECTATION_MET`: activity falls within the policy's stated window and expectation. More activity is not inherently better.

Each predicate returns `PASS`, `FAIL`, `UNKNOWN`, `STALE`, `ERROR`, or `NOT_APPLICABLE` under a pinned policy. The public case study renders partial lamps, but it does not implement this general six-predicate decision contract. PROPOSED

On the consequence signal, red means `NO_BLOCKING_CONSEQUENCE = FAIL`: a named blocking consequence is active under the pinned policy. It can request acknowledgement before another scoped action begins. Acknowledgement does not make the condition safe, and some red conditions are non-waivable and require refusal. The public educational interface does not claim to control a host chat or an external tool. Without a durable, enforceable gate the indicator is informational, and I say so rather than implying enforcement.

The record separates `NOTIFIED`, `PRESENTED`, `ACKNOWLEDGED`, `AUTHORIZED`, `OVERRIDDEN`, `INTERVENED`, and `RESOLVED`. Understanding is never inferred from a click. Intervention does not prove an adverse effect was prevented. Resolution requires its own observation.

8 Methods and artifact scope

8.1 Evidence units and pinned sources

This paper is a bounded artifact audit and a single-project case study. The primary implementation evidence is pinned to commit `275d0b3e7474ef58456c82a042163567cd12122f` of the public Resilience Ledger repository. I reran its public gate on Node 24.18.0 and Python 3.14.6. Continuous integration declares Node 22.17.1 and Python 3.12.10. The same recorded projection root was produced by separate JavaScript and Python implementations derived from one specification and fixture corpus. This is cross-language replay parity, not clean-room or external replication.

The protocol suites use finite event files, policies, fixtures, and rejection mutations as their units. Their results are exact only for those bytes and that code. Deployment observations use paths or routes sampled at named times. Interface checks establish source or rendering structure and make no claim about reader comprehension. These evidence families are reported separately because their denominators are not interchangeable.

The Typed Refusal reanalysis uses a hand-decomposed claim as its unit. Its archive reports twelve frozen questions and three runs per P0 through P4 arm, plus a separate larger-model control, but publishes only pooled arm totals. The exact model version, generating prompts, answers, run-level data, question-level data, corpus bytes, and preregistration record are absent. Wilson intervals and Fisher exact tests were recomputed from `data/arms.json` by `data/stats.py`; they are exploratory claim-level summaries under a working independence assumption. The claims are clustered within questions and runs, so those intervals and p values do not establish arm-level precision or significance.

The 2026-07-17 replication declares 36 sessions, of which 30 were scored after six final-block sessions were truncated. Its registration record and decision logs are public at commit `77408db59cad3f968ac9ba5a0c0c6689a90e80d4` of `JakeTOpenSource/the-stable`, and its recorded cells were replayed offline. The inspected public history does not independently establish that the registration file predates data collection, so I treat it as a committed registration record rather than verified prospective registration. The Typed Refusal aggregates are pinned

separately at commit `721a824c9f735d3972d720b41685469a1020fa91` of `JakeTOpenSource/typed-refusal-harness`. No external evaluator selected, ran, or scored these experiments, and no qualified instrument or live consequential adapter was evaluated.

8.2 Finite activation, quotient, and blind-prompt packets

Three local owner-review packets test the newer mathematical layer. The Generic Device Activation Fixture enumerates 151 synthetic trace prefixes of length at most ten. It evaluates seven declared queries against ten candidate representation fields, exhaustively checks all 1,023 nonempty candidate subsets, and retains a collision witness whenever a representation merges records whose query answers differ. Separate Python and JavaScript generators produce the same frozen dataset; the exhaustive subset analysis is then performed in Python. The result is exact only for those traces, queries, fields, and transition rules.

The transition-stable quotient packet uses the same 151 records and ten declared events. Each state-event pair is either enabled, advancing to its child trace, or refused, remaining at the current trace. This gives 1,510 finite transitions. A partition begins from a declared query signature and repeatedly splits any class whose members differ in event status or successor class. Separate Python and JavaScript analyzers produce byte-identical canonical reports. This is a finite application of established sequential-machine refinement [43-45], not a new minimization theorem.

The blind packet froze one prompt, one response schema, a nine-group oracle, and a twelve-function semantic rubric before three responses were evaluated. The responses were requested under `gpt-5.6-sol/high`, `gpt-5.6-sol/low`, and `gpt-5.6-terra/high` configurations. Those labels are request metadata because the retained responses contain no runtime model attestation, model-build digest, seed, or sampling parameters. The agents saw the prompt and schema, not the oracle or rubric. Exact fields were compared with the frozen oracle. Semantic recurrence was mapped separately and required a verbatim quote from the corresponding response. That map remains `DRAFT_OWNER_REVIEW`.

The packets share an operator, orchestration platform, prompt, response schema, and likely model ancestry. Agreement is therefore a bounded output-recovery result, not independent validation. The expected activation analysis, quotient report, and blind report are pinned locally by digests `7c550d125d38`, `1b0e78adcac7`, and `de2c28735762`. Appendix C gives the full values and paths. **TESTED**

8.3 Related work boundary

The design joins established lines of work rather than treating their components as new. Causal ordering and state-machine replication [2, 6], event sourcing and transaction or compensation boundaries [3-5], runtime assurance [7], metrology and conformity assessment [8, 9, 11], and reproducibility, estimands, and causal inference [10, 12, 13] provide the main technical background. Canonicalization, transparent logs, provenance, and software supply-chain records inform the evidence identity boundary [15-20]. Accessibility, assurance-case, status-condition, and interchange specifications inform the reporting surface [18, 27, 31-34].

Proper scoring and empirical forecast calibration supply the probabilistic lane [39-41]. Performative prediction supplies the warning that a decision can change the distribution it is later scored against [42]. Sequential-machine equivalence and partition refinement supply the finite transition-stable construction [43-45]. Convex and vector optimization supply the distinction between a Pareto frontier and a policy-selected point [46].

Robust convex optimization supplies the worst-case-over-a-declared-uncertainty-set pattern [48]. Mathlib's pinned `Function.FactorsThrough` definition supplies the formal vocabulary for query-relative sufficiency [47]. These are established sources used to express the proposal; none validates the case study.

Privacy, security, financial, medical-device, and AI-governance sources are used as domain constraints or comparison points [14, 21-26, 28-30]. They do not establish compliance. The transformer, biological-mechanics, and side-channel sources supply limited architecture or measurement context [1, 35, 36], not validation of this protocol.

Hamilton-Zero makes one scientific boundary concrete. Its architecture analytically preserves a variational upper bound, while the authors warn that a finite-sample Monte Carlo estimate can appear below the true ground-state energy because of estimator noise or mixing bias [38]. A guarantee on the represented state therefore does not automatically attach to the sampled estimate or the published comparison. This is a domain example, not validation of this protocol.

With his permission, Jake Macdonald's OpenGoldenRatio (OGR) v0.1 is cited as parallel related work [37]. After reviewing this draft, Macdonald helped sharpen the comparison: STP follows governed transformation from candidate output toward accepted state, while OGR centers containment and governed relations among actors or agents. His contribution here was review and clarification of that comparison. He did not contribute code, data, experiments, or authorship, and OGR is not evidence that STP works.

9 Results

Five result families are kept apart because their denominators and their meaning differ. Protocol conformance yields exact finite outputs. Agent behavior yields bounded empirical observations. Interface work yields structural conformance and no comprehension claim. Live operations yield bounded observations at named times. Finite mathematical packets yield exact local results over declared traces, query sets, candidate fields, transition semantics, and prompt-oracle comparisons.

9.1 The gate

One command runs the public suite. It executes sixteen scripts and prints thirty-four numbered holds across twelve named suites, with every denominator equal to its numerator.

Table 6. Public gate composition at commit `275d0b3e7474`, from `node governance/harnesses/run-all.js`.

Suite	Holds	What the strongest hold in it establishes
Ledger falsification	10	Ten mutations of the event history are rejected
Governance chain	5	16 event files, 6 stream chains, 12 checkpoints bind
Authority falsification	4	Four forged authority paths are rejected
Cross-language replay parity	3	JavaScript and Python implementations derived from one specification and fixture corpus produce the same root
Append-only history	2	Git comparison permits additions only

Suite	Holds	What the strongest hold in it establishes
Privacy boundary	2	85 records scanned, 3 synthetic leak canaries caught
Atlas data sync	2	Six projections match baseline, 4 mutations rejected
Six-signal public surface	2	Six conditions render with non-color cues
Schema contract	1	Schemas, validators, and envelopes agree
Atlas data materialization	1	Three profiles replay, three malformed inputs rejected
Atlas foundational repair	1	The repaired foundation still holds
Authority	1	The authority profile evaluates its seven conditions
Twelve suites	**34**	**All holding at this commit**

Four of the sixteen scripts print a named pass with no numbered hold: the replay driver, the runtime check, the home surface check, and the public explanation check. Their results are therefore invisible in the total of 34. The number understates coverage and should not be read as the count of everything checked. TESTED

9.2 One source, six incompatible views

The candidate source holds 439 terms and labels every one of them reviewed. Six public projections were measured against it. The count drift was the least of it.

Table 7. Projection drift against the 439-term candidate source. **Identical** counts shared records matching on every field. **Status differs** counts shared records whose review status disagrees.

Projection	Terms	Shared	Identical	Absent	Extra	Status differs
ask-inline-data	435	435	0	4	0	258
ground-truth-inline-data	435	435	0	4	0	258
explore-inline-data	435	435	125	4	0	258
gap-check-inline-data	433	433	0	6	0	256
canon-json-projection	214	204	0	235	10	26
canon-markdown	text document: pins a canonical text digest only, with no per-term comparison					

Three findings matter more than the counts. First, the source calls all 439 terms reviewed while three projections report 258 candidate and 177 reviewed, so the authoritative label was contradicted by every consumer. Second, in four of the five comparable projections not one shared record matched on every field. Third, the canon projection contains ten identifiers with no counterpart in the source at all, which is divergent provenance rather than staleness.

The repair did not declare one file true. It registered a candidate source, measured every projection against it, stored the mismatch sets by digest, and added mutation tests. Historical regeneration stayed impossible for some consumers because their selection rules were never recorded. The 439-term source remains a candidate inventory, and no check here validates a single definition. TESTED

9.3 Source, deployment, and live bytes

The project once shipped by manual upload, which left the relation between repository and production unclear. The first recorded reconciliation compared every deployable path.

Table 8. Production observation `34bde4ec2eb4`, recorded 2026-08-11.

Measure	Value	Reading
Deployable paths checked	102	the declared set
Returned HTTP 200	102	all reachable
Missing	0	
Semantic matches	100	
Semantic mismatches	2	`index.html` and `sw.js`
Line-ending-only differences	1	`CITATION.cff`

The two mismatches were left unresolved rather than rounded away. Production served a homepage without the deferral script the repository carried, and a service worker naming cache `aaig-v84` where the repository named `aaig-v85`. The receipt also records that the reported source commit was an empty string, and that the deployment completed roughly ten minutes before the then-current main commit existed, so that commit could not have been its source. The event decision was `DEFER`. OBSERVED

A later Git-connected deployment linked provider record to merged source with an exact commit relationship, and sampled two live routes. Both returned 200. Both differed from committed bytes by exactly one declared 214-byte analytics insertion with zero source bytes removed, which is why raw byte identity is recorded as mismatched and the transform relationship as matched. Both routes recorded no content security policy header. That receipt explicitly declines to establish global edge convergence, installed cache state, accessibility, privacy, security, semantic truth, durability, or any future state. OBSERVED

The service-worker drift from the first receipt stayed open for two cache generations. A third receipt now closes it: production served bytes identical to the committed file, with both naming cache `aaig-v87`. Closing it required publishing a checkpoint, and the projection root was unchanged at `22852b5a3025`, because an observation with no effect must not advance accepted state. OBSERVED

The closure is bounded and the receipt says so. It records that the `aaig-v85` and `aaig-v86` generations were never observed in production and cannot be reconstructed, that one edge was sampled, and that installed client caches were not inspected. The process failure is the part worth keeping: an unresolved finding aged out of view for two versions because nothing scheduled its re-observation. The protocol recorded the gap faithfully and did not close it for me. OPEN

9.4 Finite representations and blind output recovery

Table 9. One layer in plain language, formal language, finite result, and claim ceiling. Every result is local to the retained owner-review packet.

Plain statement	Formal object	Finite result	Claim ceiling
A forecast is not permission.	`execute = 1` only if `g = PASS`, for every `p`.	All three blind responses returned `BLOCK` when the gate was held.	Exact prompt-oracle agreement, not operational enforcement.

Plain statement	Formal object	Finite result	Claim ceiling
In the frozen objective, coupling the report to its reward moves the optimum.	`argmin E[(p-Y)^2] = 1/2`; adding `(1/2)p` gives `p* = 1/4`.	All three responses recovered both frozen values.	A synthetic algebraic counterexample, not real-world calibration.
Current state is sufficient only for named questions.	`r(x)=r(y)` implies `sigma_Q(x)=sigma_Q(y)`.	Across 151 traces and seven queries, all 1,023 nonempty subsets of ten fields were checked. One five-field set was sufficient. It realized 47 tuples for 33 query classes.	Set-minimal within ten supplied fields, not globally minimal. The 47 tuples overrefine the exact 33-class query quotient.
A useful summary must also survive permitted next steps.	`x equiv_Q y` only when every permitted continuation preserves equal query answers.	The full seven-query partition stayed `33 to 33`. Removing `nextPermittedActions` began at 18 classes and refined to the same 33-class partition in one round.	Exact for one finite graph, ten events, and declared refusal semantics.
Several actions can remain equally admissible without being equal.	Keep every nondominated risk vector until policy supplies a preference rule.	All three responses retained `A, B, C` as Pareto-minimal and refused to invent a unique action.	Agreement on the frozen example, not a universal risk policy.

The blind evaluator made 27 exact comparisons: nine frozen result groups across three requested configurations. All 27 matched the oracle, all three response shapes passed, and the exact answer vectors matched pairwise. The exact layer includes the gate, the two forecast optima, historical insufficiency, the Pareto set, absence of a unique action, the unresolved pending state, the encoding distinction, and the five-step record order. TESTED

The semantic layer is deliberately weaker. Its quote links pass deterministic existence checks, but the function-to-quote judgment remains `DRAFT_OWNER_REVIEW`. Six functions have unambiguous unanimous quote support: gate and forecast separation, freezing before resolution, append-only resolution, cohort calibration, typed unresolved state, and preservation of a Pareto frontier without hidden scalarization. The owner-review map also marks forecast scoring, F04, present in all three responses. One mapped quote says to score the frozen forecast after resolution without naming a declared scoring rule, so strict F04 unanimity remains unresolved and is not promoted to the six-function count. F04 still has direct scoring-rule support in two responses. Query-relative projection and behavioral quotienting also recurred in two of three responses, so functions F01 through F09 each have quote support in at least two. Deterministic replay audit, explicit separation of belief scoring from action optimization, and the general claim ceiling, F10 through F12, were absent from all three. Agreement is therefore signal about recoverable output structure, not evidence that the responses supplied the complete architecture. OPEN

The model labels are retained exactly as requested but not promoted to runtime identity. The runs share material common causes, including the prompt, schema, orchestration platform, and possible training or system dependencies. No result in this subsection is described as independent replication, human understanding, truth, novelty, safety, or forecast calibration.

10 The results that went against me

These are the most informative findings in the project. Each one narrowed a claim I had already made.

10.1 A receipt that contradicts itself

The first deployment receipt reports its status twice. The envelope records `evidence: VERIFIED` and `authority: UNVERIFIED`. The payload record inside the same file reports `evidence: PASS` and `authority: PASS_WITH_LIMITS`. Five other axes agree. Two do not, and they disagree about whether authority was established.

The schema contract gate passes this file. It validates each object against its own schema and never cross-checks the two. So a receipt can be internally inconsistent on the question of whether anything was authorized, and a green gate will not notice. This is exactly the projection drift the design warns about, occurring inside a single artifact of the system that names it. TESTED

A second instance sits beside it. The two deployment receipts use different status vocabularies. The first is schema 1.0.0 with no declared vocabulary and pass-and-fail values. The second is schema 2.0.0 declaring `stp-v1.1-status-axes` with values such as `SUPPORTED`, `APPLIED`, and `MATCHED`. No mapping between them exists in the repository, so the two production observations in one stream cannot be compared axis by axis. OPEN

10.2 A larger-model control recorded fewer unsupported claims than every eligible structured arm

The Typed Refusal archive reports unsupported-claim aggregates across five arms of increasing structure, against a corpus of United States Code Title 29 identified by digest `188ab1c50a46`. A sixth arm, an off-model control, was a larger model given the corpus and no structure at all. The corpus bytes and original runs are not in the public archive.

Table 10. Typed Refusal Harness. Rates are unsupported claims per 100 hand-decomposed claims. Wilson intervals and two-tailed Fisher exact tests against P0 are exploratory claim-level summaries under a working independence assumption; question and run clustering could not be modeled from the published aggregate.

Arm	Structure added	Unsupported	Claims	Rate	95% CI	Exploratory p vs P0
P0	corpus only, no index, no tool	20	90	22.2	14.9-31.8	baseline
P1	hash-verified snapshot, single-unit pull	5	87	5.7	2.5-12.8	0.0021
P2	typed rejections as final answers	6	105	5.7	2.6-11.9	0.0012
P3	byte receipt required per quotation	9	100	9.0	4.8-16.2	0.0148
P4	frozen answers with inline receipts	0	99	0.0	0.0-3.7	excluded
Control	**larger model, corpus only, no structure**	**0**	**151**	**0.0**	**0.0-2.5**	**not tested**

At the claim level, the archived aggregates yield p = 0.0021 for P1, p = 0.0012 for P2, and p = 0.0148 for P3 against P0. No decision threshold was registered, and the independence assumption is not supported by the clustered design, so these values are not treated as confirmatory or as arm-level significance tests. P4's zero count is descriptive only. Its answers were supplied by construction, and one of its three runs ignored the cards, so the arm is excluded from accuracy claims.

The larger-model control recorded zero unsupported claims out of 151, matching the excluded P4 count and recording fewer than each eligible structured arm, P1 through P3. Exploratory claim-level Fisher comparisons yield p = 0.0061 against P1, p = 0.0044 against P2, and p = 0.0002 against P3. Because model identity and scaffolding changed together, these comparisons do not identify a causal effect. Within the published aggregate, replacing the model coincided with a lower unsupported-claim count than any eligible scaffold around the weaker model, while P1 through P3 each remained below that weaker model's P0 baseline. OBSERVED

What this experiment does not support

The generating prompts, the per-run answers, the corpus file, and the preregistration artifact are all absent from the repository. The repository states that six predictions were registered before any arm ran and that three were falsified, and exactly one of the six is quoted anywhere, partially. I could recompute the published aggregate from `arms.json` and `stats.py`. I could not reproduce a single original run. No significance criterion was preregistered, so every p value here is post-hoc. The archive also does not publish the question-level or run-level counts needed for a cluster-preserving permutation, bootstrap, or multilevel analysis. OPEN

10.3 A committed registration record and a replication that denied its own doctrine

A separate experiment is described by its repository as preregistered. The pinned repository contains a registration file naming three criteria and decision logs for a test of the claim that live per-probe feedback eliminates the premature nulls that committed plans produce. The design was two rounds by three models by three replicates by two arms, for 36 sessions. The inspected public history does not independently prove that the registration file predates those sessions.

Table 11. Replication of 2026-07-17. Verdict: doctrine denied. Token counts are block totals of output tokens over six sessions per cell group.

Model	One-shot	Iterative	Ratio	Scored result
claude-opus-4-8	5,567	35,152	6.3×	6/6 clean one-shot; 1 premature null iterative
claude-sonnet-5	15,673	48,800	3.1×	12/12 scored as calibrated under the experiment rubric
claude-haiku-4-5	38,809	41,399	1.1×	3 clean, 3 premature one-shot; iterative arm lost

Recorded criterion (a) was satisfied, though not by the model that motivated the doctrine. Recorded criterion (b) failed, and that failure denied the doctrine: one opus iterative session produced a genuine premature null, skipping the domain floor extreme after seven matching probes sat in front of it. The recorded bar was zero counterexamples, and one is enough.

Recorded criterion (c) could not be evaluated at all. All six haiku iterative sessions were truncated mid-play by a session limit, so 30 of 36 sessions were scored. The repository record acknowledges the confound rather than hiding it: models ran in sequential blocks with haiku last, so budget exhaustion clusters on the final block. That is missing data with a known mechanism, recorded as missing. OBSERVED

Two things survived. Sonnet was scored as calibrated under that experiment's rubric in 12 of 12 sessions across both arms, which the document itself downgrades to a rubric-specific signal rather than a capability benchmark. That label is not empirical forecast calibration as defined in section 5 and is not a protocol-calibrated predicate. And all four valid premature nulls fell on the same round, with zero on the other across its 15 valid sessions, which points to a blind-spot family that crosses models and that per-probe feedback did not close.

The replay of all 36 recorded cells runs offline through the published harness, asserts twelve checks, and is wired into the repository gate. It reproduces the denied verdict from the recorded artifacts; it does not reproduce the original model sessions or constitute independent validation. TESTED

10.4 Smaller corrections

- Absolute privacy language on the public site exceeded what the tests covered. The page said nothing leaves while the host loaded analytics. The claim was split into local input analysis and aggregate page telemetry.
- The offline cache list omitted JavaScript that cached tools required, so a fresh offline profile could hold a page without the code to run it. The list was closed over its dependencies and installation became fail-closed.
- A data-only overlay still carried HTML through the parser into a rendering sink. Escaping plus a full parser-to-sink test closed the demonstrated path.
- The privacy scanner reports 85 records. The tree holds 87 files of the scanned types, and the harness hard-codes two self-exemptions. The number is correct and the exemptions are worth stating.
- The home surface check confirms the string `160 recorded cross-domain primitives` appears in the page. The data file does contain 160 entries, but the check is a string match, not a cross-count, and would pass if both drifted together.

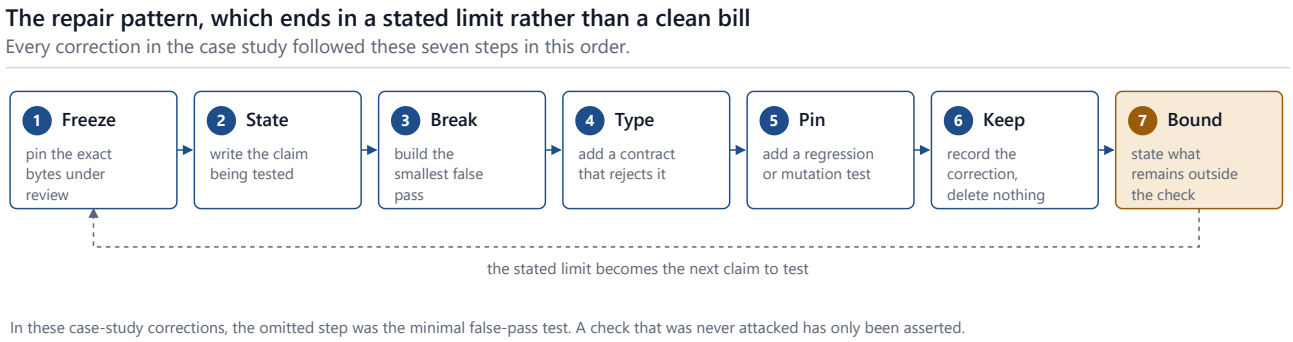

Figure 5. The repair pattern used in the corrections reported above. In these cases, the missing step was a regression gate.

11 What this does not establish

Stated once, in full, so that no section has to hedge itself.

Nothing here establishes that a recorded event was true. A false sensor produces a well-formed receipt. A valid credential holder makes a bad decision. Two programs agree because they share one mistake. Comparison against a previously trusted digest reveals a byte difference without showing the earlier bytes described reality.

Nothing here establishes causation, lawful authority, regulatory compliance, statistical reliability, general safety, or independent validation. The reducers share a specification and may share its errors. Most fixtures are synthetic. Most witnesses are not organizationally independent. There is no live authority service with independently managed keys, trusted time, revocation, and atomic single-use consumption. There is no durable non-equivocating log for a threat model that includes full history replacement. There is no general proof of liveness, fairness, concurrency safety, or survivability. There is no preregistered human study, and no external replication of the architecture.

The case study is one project's repair history, produced by one person, largely in one computing environment, on data and interfaces that changed while the work proceeded. It may not transfer.

The activation and quotient results are exhaustive only inside a synthetic finite model. Their minima depend on the supplied candidate fields and declared queries. Their stable partition depends on the 151 trace prefixes, ten events, enabled-or-refused transition rule, and finite continuation graph. They establish no fact about an iPhone, another device, an open environment, an unmodeled event, or a richer query. A five-field sufficient representation is not the unique data structure for the behavior, and its 47 realized tuples are not the exact 33-class behavioral quotient.

The blind prompt is a three-response output-recovery check, not a model benchmark. Requested model labels are unattested metadata. The runs share the prompt, schema, platform, operator, and possible training or system dependencies. Twenty-seven exact oracle matches do not establish semantic understanding. The semantic map is an owner-review judgment over quote-linked text, and its three universal absences are part of the result. No real event resolved, so the packet contains neither a forecast outcome nor evidence of forecast calibration.

The local Lean source states query-signature sufficiency and kernel exactness using Mathlib's pinned `Function.FactorsThrough` vocabulary [47]. The module compiled directly in the local pinned environment, but it is not imported by the package root and no upstream Mathlib review occurred. It is a local formalization aid, not an accepted library contribution or external proof review.

One risk deserves naming on its own. Strict preservation of unknowns can make a system unusable. If unresolved evidence blocks every action, availability and safety trade against each other, and the protocol offers no principled exchange rate between them. OPEN

Five tests that would demote these claims

1. **Clean-room replay.** Give an external team the minimized public packet and nothing else. Disagreement demotes the replay claim or exposes a hidden dependency.
2. **Formal non-promotion check.** Model the lifecycle and either prove or refute that proposal, acknowledgement, and unresolved evidence cannot advance accepted state.
3. **Qualified instrument pilot.** Use one real instrument with a metrology review, operating limits, uncertainty, and a known sensing footprint. Failure narrows the observation contract.

4. **False-assurance study.** Pre-register a comparison between one composite status and the six-signal view, measuring correct intervention, missed danger, false reassurance, and response time. No benefit leaves Six Signals an accessibility design and not a comprehension improvement.
5. **Narrow live adapter.** Implement one bounded consequential tool end to end. Any unrecorded or duplicated effect falsifies the finality boundary.

12 Adapting this

The names here do not matter. The separation does. Nine steps, in order, and the first one is the one people skip.

1. **Pick one consequential transition.** Not an ontology. One action whose wrong execution or false acceptance would actually hurt.
2. **List what you currently collapse.** Write the exact phrases your system treats as success: request accepted, job started, HTTP 200, database commit, sensor value, human review, deployment succeeded, customer outcome. Decide which are genuinely different states.
3. **Type absence.** Define what a real zero means in your domain, then define no-change, missing measurement, censoring, staleness, and evaluator failure separately. Never let an empty field pick between them.
4. **Bind authority to an operation.** Not to a role, and not to a tool. Separate risks a user may accept from constraints that must refuse. Record expiry, revocation, and replay behavior.
5. **Build the smallest reducer.** Rebuild accepted state from the event prefix using assigned sequence and causal references. Keep presentation and telemetry derivative. Write a second implementation if the state is load-bearing.
6. **Attack it.** Change a subject identifier. Duplicate an event. Remove a blocking check. Replace a source digest. Reorder events. Force an evaluator error. Return an acknowledgement with no effect. Keep every successful attack as a regression test.
7. **Report a vector.** Show consequence, evidence, integrity, privacy, activity, and the local complete condition separately, with reason codes and source links. Never color alone.
8. **Release less than you collected.** Allowlist a public derivative. Keep sensitive evidence in a restricted store with retention rules. Record what the public verifier can and cannot recreate.
9. **Invite a clean-room challenge.** The useful first external test either matches your bounded result or finds your instructions underspecified. Both results are worth more than another internal pass.

13 What I do not know

I do not know whether this combination is novel in an academic sense, and I have not completed the systematic literature review needed to assess it, so I make no priority claim. I do not know whether six signals are understood better than one status, because I have run no user study. I do not know whether strict preservation

of unknowns is affordable in a high-volume system. I do not know how any of this behaves under network partition, adversarial witnesses, or fast schema churn. I do not know whether the missing Ledger bytes would resolve the terminology conflict in section 2 or deepen it.

Those are part of the result. The clearest implementation finding in this work is narrow: specific false-pass paths became explicit tests, and unresolved conditions stayed visible instead of being rounded to green. The next tests are external reproduction, one real qualified instrument, and one bounded live adapter.

14 Lineage, credits, and AI-assistance disclosure

Where this came from

The ideas in this paper did not start here, and they did not start with me alone. The early conceptual work was done in April and May 2026 in extended dialogue with language models, principally Claude and Gemini. I set the problems, argued with the answers, and kept what survived. What the models contributed was real and I am not going to describe it as tooling.

I published the first versions publicly on LinkedIn in May and June 2026, before any of the software described here existed. Those posts introduced most of the vocabulary this work still runs on: the biological floor, the metabolic veto, agentic drift, structural harmonics, the structural floor, hidden actualities, mechanical psychosis, and state-delta architecture. Several arguments in section 1 appear there first, in less careful form. The posts are on my public profile at `linkedin.com/in/jake-tiller-548b409b`, and per-post locators belong in this paragraph once they are archived independently rather than cited from a platform that can change them.

What this paper adds to that earlier work is not the ideas. It is the part that can be checked. The posts asserted a framework. This document reports what happened when I built it, tested it, tried to break it, and recorded the places it failed. The move from assertion to evidence is the whole contribution, and the earlier material is stronger for having been narrowed by it.

I make no originality claim over the component ideas. Causal ordering, event sourcing, compensating transactions, safety and liveness, measurement uncertainty, conformity assessment, and provenance modeling are all established fields, cited in section 15, and none of them are mine. The synthesis is what I did. Whether that synthesis is novel in an academic sense requires a systematic literature review I have not completed, so I do not claim priority over anyone.

Credits and disclosure

I supplied and classified the source artifacts, set the operating, acceptance, and privacy constraints, chose which claims to make public, and am responsible for the manuscript and every release decision.

Generative AI systems were used as research, coding, testing, and editing tools. Recorded uses include brainstorming, terminology extraction, source discovery, repository inspection, code drafting, test generation, adversarial review, and manuscript editing. Their outputs were treated as candidate material, never as evidence, authority, authorship, or independent validation. Checks run by agents that share models, prompts, tools, or specifications are not described anywhere in this paper as independent replication. The synthesis and the prose benefited materially from that assistance, and I reviewed the final text.

15 References

1. A. Vaswani et al. Attention Is All You Need. NeurIPS, 2017. papers.nips.cc/paper/7181
2. L. Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. CACM 21(7), 1978. doi:10.1145/359545.359563
3. M. Fowler. Event Sourcing. 2005. martinfowler.com/eaaDev/EventSourcing.html
4. J. Gray. The Transaction Concept: Virtues and Limitations. VLDB, 1981.
5. H. Garcia-Molina and K. Salem. Sagas. SIGMOD, 1987. doi:10.1145/38713.38742
6. F. B. Schneider. Implementing Fault-Tolerant Services Using the State Machine Approach. ACM Computing Surveys 22(4), 1990. doi:10.1145/98163.98167
7. D. Seto et al. The Simplex Architecture for Safe On-Line Control System Upgrades. ACC, 1998. doi:10.1109/ACC.1998.703255
8. JCGM 200:2012. International Vocabulary of Metrology, 3rd ed. doi:10.59161/jcgm200-2012
9. JCGM 106:2012. The Role of Measurement Uncertainty in Conformity Assessment. doi:10.59161/jcgm106-2012
10. National Academies. Reproducibility and Replicability in Science. 2019. doi:10.17226/25303
11. JCGM 100:2008. Guide to the Expression of Uncertainty in Measurement.
12. ICH E9(R1). Estimands and Sensitivity Analysis in Clinical Trials. Final addendum.
13. M. A. Hernán and J. M. Robins. Causal Inference: What If. 2020. miguelhernan.org/whatifbook
14. EU General Data Protection Regulation, Articles 5 and 17. Regulation 2016/679.
15. A. Rundgren, B. Jordan, S. Erdtman. JSON Canonicalization Scheme. RFC 8785, 2020.
16. B. Laurie et al. Certificate Transparency Version 2.0. RFC 9162, 2021.
17. W3C. PROV-DM: The PROV Data Model. Recommendation, 2013.
18. W3C. Web Content Accessibility Guidelines 2.2. Recommendation, 2024.
19. S. Torres-Arias et al. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. USENIX Security, 2019.
20. SLSA. Supply-chain Levels for Software Artifacts, v1.2. slsa.dev/spec/v1.2
21. NIST SP 800-207. Zero Trust Architecture. 2020.
22. NIST SP 800-82 Rev. 3. Guide to Operational Technology Security. 2023.
23. NIST AI 100-1. Artificial Intelligence Risk Management Framework 1.0. 2023.
24. CPMI-IOSCO. Principles for Financial Market Infrastructures. 2012.
25. U.S. SEC. Risk Management Controls for Brokers or Dealers With Market Access. Rule 15c3-5.
26. BCBS 239. Principles for Effective Risk Data Aggregation and Risk Reporting. 2013.
27. OMG. Structured Assurance Case Metamodel, v2.3. 2023.
28. U.S. FDA. Predetermined Change Control Plan for AI-Enabled Device Software Functions. 2025.
29. U.S. FDA. Applying Human Factors and Usability Engineering to Medical Devices. 2016.
30. EU Artificial Intelligence Act, Regulation 2024/1689, Articles 12, 14, 19.
31. Kubernetes. KEP-1623, Standardize Conditions. kubernetes.dev/resources/keps/1623
32. CNCF. CloudEvents Specification 1.0.2. 2022.
33. JSON Schema. Core and Validation, Draft 2020-12.
34. Model Context Protocol. Specification revision 2026-07-28.
35. L. Marom, S. Tibbits, G. Zardini, M. J. Buehler. A Category-Theoretic Framework from Biological Mechanics to Engineered Stimulus-Response Systems. arXiv:2604.26367, 2026.
36. P. Kocher, J. Jaffe, B. Jun. Differential Power Analysis. CRYPTO, 1999. doi:10.1007/3-540-48405-1_25
37. J. Macdonald. OpenGoldenRatio (OGR) v0.1: Containment-First Multi-Agent Governance Protocol. Zenodo, 2026. doi:10.5281/zenodo.18969396. Executable demonstration at commit `58450185582f4ecf1410b33f77e22d8d4b0441a2`.

38. T. Heightman, E. Orlova, P. Mantrov, and A. Ustimenko. Hamilton-Zero: A Neural Tensor-Network Foundation Model for Ground States of Arbitrary Quadratic Qubit Hamiltonians. arXiv:2608.11911v2 [quant-ph], 2026. doi:10.48550/arXiv.2608.11911.

39. G. W. Brier. Verification of Forecasts Expressed in Terms of Probability. Monthly Weather Review 78(1), 1950. doi:10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

40. T. Gneiting and A. E. Raftery. Strictly Proper Scoring Rules, Prediction, and Estimation. Journal of the American Statistical Association 102(477), 2007. doi:10.1198/016214506000001437.

41. A. P. Dawid. Calibration-Based Empirical Probability. Annals of Statistics 13(4), 1985. doi:10.1214/aos/1176349736.

42. J. C. Perdomo, T. Zrnic, C. Mendler-Dünner, and M. Hardt. Performative Prediction. Proceedings of Machine Learning Research 119, 2020. proceedings.mlr.press/v119/perdomo20a.html.

43. E. F. Moore. Gedanken-experiments on Sequential Machines. In Automata Studies, 1956. doi:10.1515/9781400882618-006.

44. A. Nerode. Linear Automaton Transformations. Proceedings of the American Mathematical Society 9(4), 1958. doi:10.1090/S0002-9939-1958-0135681-9.

45. J. E. Hopcroft. An n log n Algorithm for Minimizing States in a Finite Automaton. Stanford CS-TR-71-190, 1971. i.stanford.edu/TR/CS-TR-71-190.html.

46. S. Boyd and L. Vandenberghe. Convex Optimization. Cambridge University Press, 2004. web.stanford.edu/~boyd/cvxbook.

47. Leanprover-community. Mathlib `Function.FactorsThrough`, pinned at commit `520045ab14e26149ee970e2e617ca04b09bde5d6`. Mathlib/Logic/Function/Basic.lean, lines 832-885.

48. A. Ben-Tal and A. Nemirovski. Robust Convex Optimization. Mathematics of Operations Research 23(4), 1998. doi:10.1287/moor.23.4.769.

A Properties, assumptions, and limits

A.1 Proposal non-promotion

Let `L(k)` be a valid event prefix and `A(k) = R(P, L(k))`. Let `U(P)` be the nonempty set of policy-authorized projection-update events, containing only qualified `ACCEPT` records that close a governed transition. A `CORRECT` record begins a new governed transition and may propose a superseding state, but it cannot update `A(k)` without a later qualified `ACCEPT`. Appending only records whose types lie outside `U(P)`, including proposal, preparation, tool acknowledgement, and an unaccepted correction, cannot change `A(k)`.

By induction over the appended sequence. The base projection is unchanged, and each step records history without invoking the update function. The result depends on complete reference validation and on there being no second update path. A reducer defect or an incomplete policy invalidates the assumption, and section 10.1 shows a related assumption failing in practice.

A.2 Unknown preservation

For the proposed normalized aggregate over a finite nonempty required set, the result is `PASS` only when every condition passes, `FAIL` if any fails, and `UNRESOLVED` if none fails and any is unknown, stale, or errored. An empty set returns `UNRESOLVED` with reason `INVALID_POLICY`. No unresolved required predicate produces a pass. The rule says nothing about predicates omitted from the set. This general precedence rule has not been exercised by a mixed false-plus-unknown public fixture. **PROPOSED**

A.3 Deterministic replay

With fixed policy bytes, event bytes, schema versions, canonicalization, reducer code, and deterministic dependencies, repeated evaluation returns the same projection. This is a property of the computational boundary. It does not establish that the events are true, complete, or authorized.

A.4 Hash-link mutation detection

Assuming second-preimage resistance, an unambiguous canonical encoding, and a trusted externally anchored tip that transitively commits the event, modifying that event changes the committed tip except with negligible probability. A checkpoint protects only the prefix it commits. An anchor before a modified event does not prevent changing a later event and rehashing the suffix, so detecting suffix replacement requires an authenticated current tip.

A.5 Illustrative effect-trace counterexample

Consider two declared effects, `alpha` and `beta`. Schedule `(alpha, beta)` emits the ordered trace `[dispatch-alpha, dispatch-beta]`, while schedule `(beta, alpha)` emits `[dispatch-beta, dispatch-alpha]`. If both schedules reduce to the same accepted projection, equal projections still do not entail equal ordered effect traces. This is a counterexample by construction at the specification level. No public fixture in the pinned repository implements it, so it is not a tested result. PROPOSED

A.6 Bounded lane balance

Capacity is tracked per lane, and the three lanes do not share a unit. Observation, settlement, and recovery each measure something different, so their backlogs are held as a vector and never summed. For a declared lane `x`, a finite trace is evaluated by the deterministic recurrence:

```
B_x[k+1]  =  max( 0,  B_x[k] + A_x[k] - S_x[k] )

M_x(H)        =  max { B_x[k] : 0 <= k <= H }

finite_capacity_pass_x(H)  iff  M_x(H) <= C_x
```

A_x arrivals into lane x · **S_x** service capacity of lane x · **B_x** backlog · **C_x** declared finite capacity · **H** declared finite horizon · all lane quantities finite, nonnegative, and in one declared unit. PROPOSED

For declared arrays, initial backlog, horizon, and capacity, these equations answer one bounded question: whether the computed backlog exceeds capacity anywhere in that finite trace. They do not establish stationarity, asymptotic stability, recurrence class, a queue-length distribution, or a future arrival or service rate. No stochastic queueing theorem is claimed or tested here.

The autonomy rule in section 3 treats observation, settlement, and recovery capacity as separate constraints. Applying it to a live lane would require declared measurement procedures and a justified rule for projecting beyond the observed window. This project supplies neither. A finite-capacity pass is therefore a local trace result, not evidence that a live lane will keep pace. OPEN

Where a bounded check is wanted before an estimator exists, the survivability harness substitutes finite reachable-state traversal at a declared horizon. The profile fixes `H = 7` rounds and a no-change tolerance of `epsilon = 0.02` in the lane's declared unit. Both are conventions. A longer horizon evaluates a different, generally more expensive bounded question, and neither value is derived from anything. Under an exact-`H` recovery condition, one horizon is not uniformly stronger or weaker than another without additional monotonicity and absorbing-target assumptions. Every disturbed and controlled state must stay legal, preserve the named invariant, and retain the required essential function, and every state in the frontier at round `H` must

be in the recovery target set. Merely reaching the target before `H` is insufficient unless the required `H`-frontier condition also holds. An invalid model, an undefined controller, or an exceeded bound returns `UNKNOWN` rather than a pass. **PROPOSED**

A.7 Brier loss and the coupled-objective shift

Let `Y` be binary with `Pr(Y=1)=q`. For a forecast `p`, direct expansion gives:

```
E[(p-Y)^2]
  = q(p-1)^2 + (1-q)p^2
  = p^2 - 2pq + q
  = (p-q)^2 + q(1-q)
```

The final term is constant in **p**, so the unique minimum on the unit interval is **p = q**. This is the binary Brier result [39, 40].

If the same objective adds `mu p`, its derivative is `2(p-q)+mu`. Strict convexity gives the constrained minimizer `clip(q-mu/2, 0, 1)`. At `q=1/2` and `mu=1/2`, the optimum moves from `1/2` to `1/4`. The lower report is not a better estimate of `q`. It is the optimum of a different objective. The counterexample proves that an incentive attached directly to the report can distort the report; it does not prove that every coupled system does so.

A.8 Query factorization and exact kernels

For a finite family of queries, let `sigma_Q(x)` be the vector of all declared answers at state `x`, and let `r(x)` be a proposed representation. The representation is sufficient exactly when equal represented values never hide unequal query signatures:

```
r(x) = r(y)  implies  sigma_Q(x) = sigma_Q(y)

equivalently,  sigma_Q = d after r  on the image of r
```

This is `sigma_Q.FactorsThrough r` in Mathlib's pinned vocabulary [47]. The decoder **d** need only be defined on represented values that occur.

Exactness requires the reverse factorization as well. Then `r(x)=r(y)` if and only if `sigma_Q(x)=sigma_Q(y)`, so the two functions induce the same kernel partition. Their class labels and data structures may still differ. The local `QueryQuotient.lean` module proves component factorization, the sufficiency equivalence, separation of unequal signatures, and this kernel characterization. It compiled directly against the pinned Mathlib environment but is not an upstream-reviewed contribution.

In the finite activation packet, the unique five-field candidate minimum is sufficient for all seven queries over 151 traces. It realizes 47 representation values, while the full query signature realizes 33 classes. It therefore preserves the answers but does not implement the exact quotient. The result is relative to the supplied ten fields and exhaustive 1,023-subset search.

A.9 Future-stable refinement

Static query equality is the initial relation `x equiv_0 y` when `sigma_Q(x)=sigma_Q(y)`. Define the next relation by retaining a pair only when it was previously equivalent and every declared event has the same enabled-or-refused status and leads to states equivalent under the previous relation. Each round can split classes and never

merge them. On a finite state set the descending sequence must stabilize.

At the fixed point, equivalent states have the same query answers after every permitted finite continuation. Conversely, any transition-stable equivalence lying inside the initial query kernel survives every refinement round by induction, so it lies inside the fixed point. The limit is therefore the largest transition-stable equivalence contained in the declared query kernel, or the coarsest stable refinement of its partition. This is established sequential-machine refinement, not a new theorem [43-45].

The frozen packet's full seven-query partition began with 33 classes and was already stable. Omitting `nextPermittedActions` began with 18 classes and refined to 33 in one round. Both reached the same partition digest `2f129b2ac6c0`. The witness is concrete: after `BOOT`, `ACCEPT_CONSENT` is enabled and advances; from the empty trace it is refused and remains in place. This proves the distinction only in the pinned finite graph.

A.10 Pareto existence and policy selection

Let a finite nonempty action set carry a finite risk vector. Say action `a` dominates `b` when every component of `a` is no worse and at least one is strictly better. A Pareto-minimal action must exist. Start from any action. If it is dominated, move to a dominator. Strict dominance cannot cycle, and a finite set cannot support an infinite descent, so the process ends at a nondominated action.

If every scalarization weight is positive, a minimizer of the weighted sum is Pareto-minimal: a dominator would make at least one positively weighted component smaller and none larger, contradicting minimality [46]. The converse does not give one authorized weight vector, and neither existence result gives uniqueness. In the blind fixture, all three actions are nondominated. Returning the frontier and an unresolved selection is therefore the complete result until policy supplies a preference rule.

B Provenance, reuse, and attribution

The intended public release will use CC BY 4.0. Adaptation will be welcome, including commercial adaptation, with attribution to the author and identification of what was modified. I cite my own sources throughout and expect the same in return, which is the whole of what I am asking.

For publication, the exact release bytes will be hashed with SHA-256, recorded in the public event history described in section 6, sealed by a checkpoint, and checked again in continuous integration. Until that workflow runs against the final release, this owner-review draft has no completed publication commitment. Once complete, the record can support artifact identity and chronology for the committed bytes. It does not by itself prove authorship, originality, independent creation, or legal priority.

Interoperability fixtures in this specification

Several values here are arbitrary by construction, meaning any distinct value would serve the same technical purpose. They are fixed so that implementations can exchange and replay the same records. They are technical fixtures, not watermarks or evidence of origin:

- the domain separation tag `D` in section 6;

- the condition vocabulary `PASS | FAIL | UNKNOWN | STALE | ERROR | NOT_APPLICABLE`, and the reason code `INVALID_POLICY` returned for an empty required set;
- the coined terms **protocol-calibrated predicate**, **decisive evidence coverage**, **decisive conformance**, and **non-stale status fraction**, each defined at first use;
- the three-lane capacity vector of A.6, naming observation, settlement, and recovery as separately metered lanes that are never summed.

An implementation that adopts this protocol may reproduce these values under the license. Attribution and identification of modifications are license obligations, separate from any technical identity check.

C Artifact index

Full SHA-256 values for every digest abbreviated in the text. The table separates public file bytes, derived outputs, and declared digests because they have different verification ceilings.

Verification note: for public blob rows, check out the named commit and hash the exact file bytes with a local SHA-256 tool. The projection-root row is reproduced by the named verifier, not by hashing that script. The refusal corpus row cannot be recomputed from the public repository because the source bytes are absent. No single command applies to every row.

Table 12. Exact locators and expected digests. Repository abbreviations are RL for Resilience-Ledger, TS for the-stable, and TR for typed-refusal-harness.

Artifact	Exact locator and status	Expected SHA-256 or root
Atlas data-sync contract	public blob: RL@ `275d0b3e7474` `governance/contracts/atlas-data-sync.contract.v2.json`	`7366c7042ec2e40a501fe091f9367eb5e8e8449763b69afadf29762864d58263`
Atlas runtime contract	public blob: RL@ `275d0b3e7474` `governance/contracts/atlas-runtime-contract.v1.json`	`7cd8c2a89f6df20995789f066643240a4cbcbc3ca67d2dc1cc4c71129b22ffd5`
Production observation 000001	public blob: RL@ `275d0b3e7474` `governance/ledger/events/deployment/000001-wp0-production-observation.json`	`34bde4ec2eb4d1bfb70b8d44df6439cb295bd1dc1293df0ae47490193ad3fa97`
Production observation 000002	public blob: RL@ `275d0b3e7474` `governance/ledger/events/deployment/000002-public-explanation-production-observed.json`	`4917a927727e3b0cc03cd057100a698ccc18e51751f69dd33f5bed14344fa24f`
STP v1.2 release manifest	public blob: RL@ `275d0b3e7474` `research/stp-v1.2/release-manifest.json`	`2f95ed233a20060d1cbca3fae555410732242b26d9fb08afe482bc3390077704`
Cross-language projection root	derived output: RL@ `275d0b3e7474` `node governance/harnesses/verify-replayers.js`	`22852b5a3025d4ed7ee1d26cc4efcd51ae2e3e02ba2a20332c2a09827d6462ca`
Refusal corpus, US Code Title 29	declared only: TR@ `721a824c9f73` `data/arms.json#corpus.sha256`; source bytes absent	`188ab1c50a46f0dd2ff32aaa5f65c759a07710e052d297644b1a8f6b58ff413d`
Replication registration record	public blob: TS@ `77408db59cad` `experiments/replication-2026-07-17/PREREGISTRATION.md`	`eff780cff6a4522370af2f00d01a7dc121ab143677f805cbdc865620dad7820b`

Artifact	Exact locator and status	Expected SHA-256 or root
Replication decision logs	public blob: TS@ `77408db59cad` `experiments/replication-2026-07-17/decision-logs.json`	`9fb48b2c0a837f91581c5faf5a043126348b6f86e64bf2383182f65978ffdca6`
Generic activation dataset	local owner-review artifact, 98,769 bytes `work/device-activation-fixture/dataset/device-activation-v1.json`	`a2dece0b00e9659e3f50df307bd41dedc722a1c5b93b16153f616d1f2b58a179`
Generic activation checker config	local owner-review artifact, 540 bytes `work/device-activation-fixture/config/checker-v2.config.json`	`7d6545f4d5cfa603b33f94ef42f747e4bf5e98631edfef30896cb5d24fb31c4d`
Generic activation analysis	local owner-review artifact, 2,737 bytes `work/device-activation-fixture/results/expected-analysis.json`	`7c550d125d383f7238ff936c7d05a3815ceb35132326253b973562fbca4b0a80`
Generic activation receipt	local owner-review receipt, 3,158 bytes `work/device-activation-fixture/evidence/BUILD-RECEIPT-000001.md`	`ee60b9aa4baa0286fb5899255380d6bf9772ca9240354bddc3b1a75fce1b9ab6`
Transition-stable quotient report	local owner-review artifact, 2,627 bytes `work/transition-stable-quotient/results/expected-report.json`	`1b0e78adcac732561a0263ef2704d53b397597c502f81e88a0999a37232df183`
Transition-stable partition over frozen case IDs	derived output: `stablePartitionSha256` in the transition-stable quotient report	`2f129b2ac6c060d253831dbded1810cfd64b030fa6b8a0514d6e048fc7086187`
Transition-stable quotient receipt	local owner-review receipt, 4,016 bytes `work/transition-stable-quotient/evidence/BUILD-RECEIPT-000001.md`	`4d0b0f50c361c51734db51a786fdc40b85de591e077d295677dbd40d63967514`
Blind prompt	local owner-private input, 1,662 bytes `work/probabilistic-audit-lane-study/PROMPT.md`	`6b0628ef41bdf3b8d871238aa39ac44af43576887d5e0b1ed44ad8e7cdeccaf1`
Blind response schema	local owner-private input, 1,808 bytes `work/probabilistic-audit-lane-study/schemas/response.schema.json`	`3656a398b63255eefc2121327da65884cca365a965f5601d6eab18e33aa0a505`
Blind run manifest	local owner-private metadata, 2,175 bytes `work/probabilistic-audit-lane-study/RUN-MANIFEST.json`	`5e104bff1ccd4cffc684667f84783a06414cfb7177da1b43717f0c90145e2f63`
Blind evaluator report	local owner-review artifact, 20,945 bytes `work/probabilistic-audit-lane-study/results/expected-report.json`	`de2c28735762a153602fc6e4bb777520c2aa3c687837e3f64b6277c459d67fe9`
Blind packet receipt	local owner-review receipt, 4,488 bytes `work/probabilistic-audit-lane-study/evidence/BUILD-RECEIPT-000001.md`	`537e8cc13e8425e53304dd22637df6d186efa4df0be0f910a747b5f78632c815`
Lean query quotient source	local source, standalone-module compile only, 3,791 bytes `work/mathlib-zero-state/ZeroState/QueryQuotient.lean`	`cfbb166202ade30abc0c79287ff8c1acf216e91a863121ade923218caede9896`